School of Computing and Information Technology
University of Wollongong

Session: **Spring 2025**
Lecturer: **Janusz R. Getta**

ISIT912 Big Data Management
Assignment 3

Published on 29 September 2025

Scope

This assignment includes the tasks related to querying a data cube, design and implementation of HBase table, querying and manipulating data in HBase table, data processing with Pig, data processing with Spark and implementation of data stream processing application.

This assignment is due on **Saturday, 01 November 2025, 7:00pm** (sharp).

This assignment is worth **20%** of the total evaluation in the subject.

The assignment consists of 5 tasks and specification of each task starts from a new page.

Only electronic submission through Moodle at:
`https://moodle.uowplatform.edu.au/login/index.php`
will be accepted. A submission procedure is explained at the end of Assignment 1 specification.

A policy regarding late submissions is included in the subject outline.

Only one submission of Assignment 3 is allowed and only one submission per student is accepted.

A submission marked by Moodle as "late" is always treated as a late submission no matter how many seconds it is late.

A submission that contains an incorrect file attached is treated as a correct submission with all consequences coming from the evaluation of the file attached.

All files left on Moodle in a state **"**`Draft(not submitted)`**"** will not be evaluated.

A submission of compressed files (zipped, gzipped, rared, tared, 7-zipped, lhzed, … etc) is not allowed. The compressed files will not be evaluated.

An implementation that does not compile well due to one or more syntactical and/or run time errors scores no marks.

Using any sort of Generative Artificial Intelligence (GenAI) for this assignment is NOT allowed !

It is expected that all tasks included within **Assignment 3** will be solved **individually without any cooperation** with the other students. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or office hours. Plagiarism will result in a **FAIL** grade being recorded for the assessment task.

Task 1 (5 marks)
Querying a data cube

Use Hive to create an internal table `TRIP` with records of individual trips, with each record containing the driver's license, the truck's registration, the trip length in kilometers, and the date of the trip.

```
create table TRIP(
 registration   char(7),
 license        char(7),
 kilometers     decimal(2),
 tday           decimal(2),
 tmonth         decimal(2),
 tyear          decimal(4) )
   row format delimited fields terminated by ','
              stored as textfile;
```

Remove a header line from a file `task1.csv` and save the file.

Populate the table by loading data from the file `task1.csv`.

(1) 0.5 mark
Implement the following query using `GROUP BY` clause with `CUBE` operator.

Find the total number of trips per driver (`license`), per truck (`registration`), per driver and truck (`license`, `registration`), and the total number of trips.

(2) 0.5 mark
Implement the following query using `GROUP BY` clause with `ROLLUP` operator.

Find the longest trip (`kilometers`) per driver (`license`) and per driver and truck (`license, registration`) and the longest trip at all.

(3) 0.5 mark
Implement the following query using `GROUP BY` clause with `GROUPING SETS` operator.

Find the shortest trip (`kilometers`) per driver (`license`) and per truck (`registration`) and per driver and year (`license, tyear`).

Implement the following SQL queries as `SELECT` statements using window partitioning technique.

(4) 0.5 mark
For each truck, list its registration number, the length of its longest and shortest trips (in kilometers), the total number of trips, and the average trip length (in kilometers).

(5) 0.5 mark
For each truck list its registration (`registration`) and all its trips (license, `tday`, `tmonth`, `tyear`, `kilometers`) sorted in descending order of trip length (`kilometers`) and a rank (position number in an ascending order) of each trip. Use an analytic function `ROW_NUMBER()`.

(6) 0.5 mark
For each driver, list its license number (`license`), total length of all his/her trips (kilometers), and the average length of all trips (`kilometers`).

(7) 0.5 mark
For each driver (`license`) and truck (`registration`) and for each trip length (`kilometers`) list the longest trip length (`kilometers`) aggregated per driver (`license`).

(8) 0.5 mark
For each truck (registration) find how the total trip length (`kilometers`) changed year by year (`tyear`). Order the results in the ascending order of years (`tyear`).

(9) 0.5 mark
For each truck (`registration`) list an average length of the current and previous trip (`kilometers`). Order the results in the ascending order of trip length (`kilometers`).

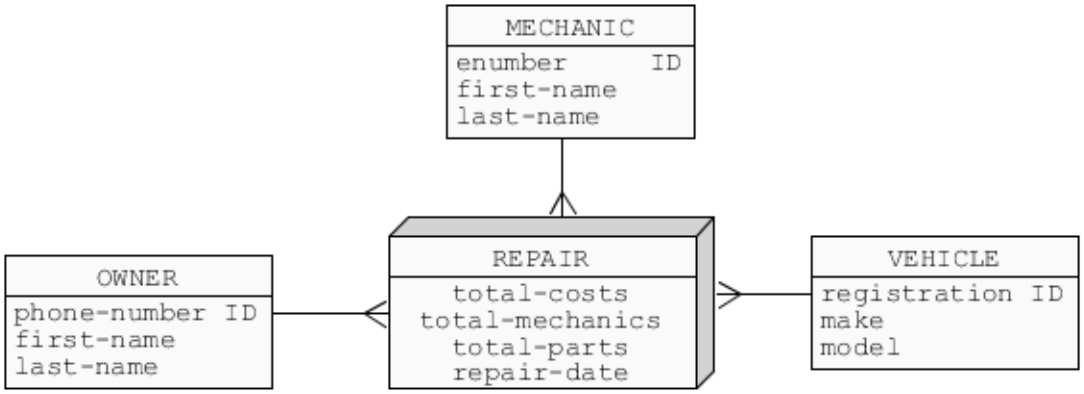
(10) 0.5 mark
For each truck (`registration`) list an average length of the current, the previous and the next trip (`kilometers`). Order the results in the ascending order of trip length (`kilometers`).

When ready, save your `SELECT` statements in a file `solution1.hql`. Then, process a script file `solution1.hql` and save the results in a report `solution1.txt`.

Deliverables
A file `solution1.txt` that contains a report from processing of `SELECT` statements implementing the queries listed above.

Task 2 (5 marks)
Design and implementation of HBase table (3 marks)

(1) Consider the following conceptual schema of a sample data cube designed to analyze vehicle repairs by mechanics for vehicle owners.



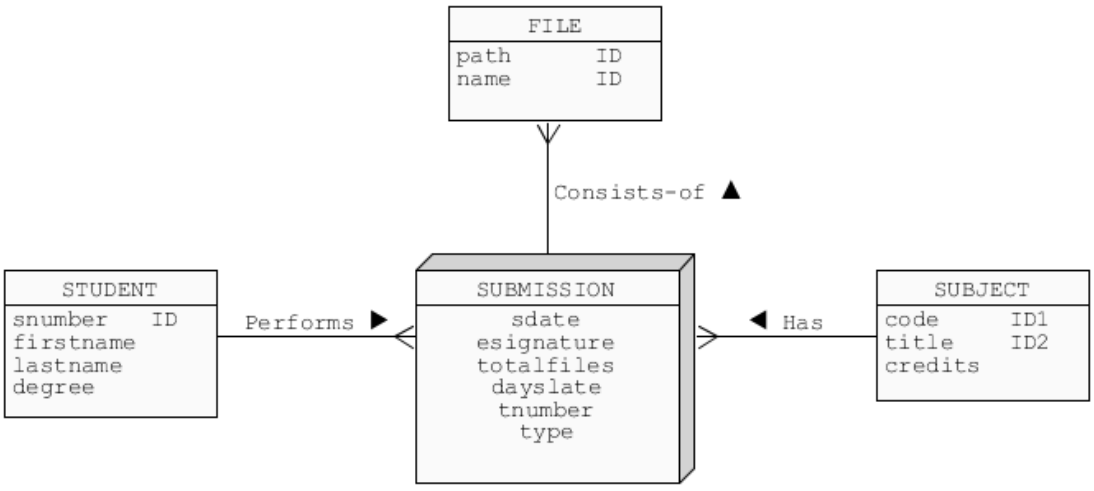
Design a single HBase table to store the data described by the conceptual schema above.

Create HBase script `solution2-1.hb` with HBase shell commands that create HBase table and load sample data into the table. Load into the table information about at least two vehicles, two owners, two mechanics and three repairs.

When ready use HBase shell to process a script file `solution2-1.hb` and to save a report from processing in a file `solution2-1.txt`.

Querying HBase table (2 marks)

(2) Consider a conceptual schema given below. The schema represents a data cube where students submit assignments and each submission consists of several files and it is related to one subject.



Download a file `task2-2.hb` with HBase shell commands. Process a script `task2-2.hb`. Processing of the script creates HBase table `task2-2` and loads some data into it.

Use HBase shell to implement the queries and data manipulations listed below. Implementation of each step is worth 0.4 of a mark.

Save the queries and data manipulations in a file `solution2-2.hb`. Note that implementation of the queries and data manipulations listed below may need more than one command of HBase shell.

(1) *Find all information included in a column family `SUBJECT` qualified by `code` and column family `FILES` qualified by `fnumber1` and `fnumber2`.*

(2) *Find all information about a subject that has a code `312`, list two versions per cell.*

(3) *Find all information about a submission of assignment `1` performed by a student `007` in a subject `312`, list one version per cell.*

(4) *Replace a submission date of assignment `1` performed by a student `007` in a subject `312` with a date `02-APR-2019` and then list a column family `SUBMISSION` to verify the results.*

(5) *Add a column family `DEGREE` that contains information about titles of degrees enrolled by the students. Assume that a student can enrol only one degree. Then add information about a title of degree enrolled by a student with a number `007`. A degree title is up to you. List all information about a student with a number `007`.*

When ready, start HBase shell and process a script file `solution2-2.hb` with the Hbase shell commands. Save report from processing of the script in a file `solution2-2.txt`.

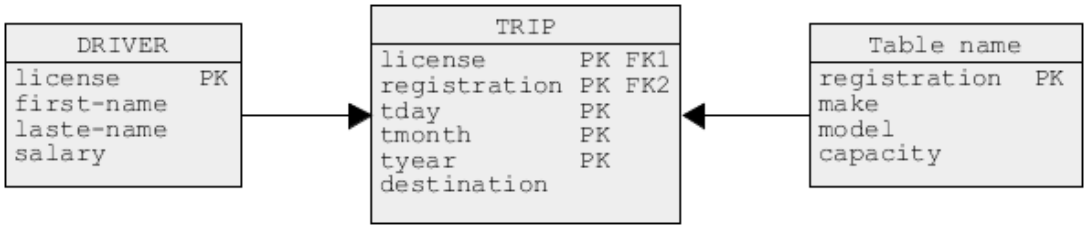
Deliverables

A file `solution2-1.txt` with a listing from processing of a script file `solution2-1.hb`.

A file `solution2-2.txt` with a listing from processing of a script file `solution2-2.hb`.

Task 3 (5 marks)
Data processing with Pig Latin

Consider the following logical schema of two-dimensional data cube.



Download a file `task3.zip` published on Moodle together with a specification of Assignment 3 and unzip it. You should obtain a folder `task3` with the following files: `driver.csv`, `truck.csv` and `trip.csv`.

Use a text editor to examine the contents of the files.

Upload the files into HDFS.

Open `Terminal` window and start `pig` command line interface to Pig. Use `pig` command line interface to implement the following actions. Implementation of each step is worth 1 mark.

(1) Use `load` command to load the files `truck.csv`, `driver.csv` and `trip.csv` from HDFS into a Pig storage.

Use Pig Latin and Pig Grunt command line interface to implement and process the following queries.

(2) Find the full names (`first-name, last name`) of drivers who used the trucks manufactured (`make`) either by `DAF` or `MAN`.

(3) Find the full names (`first-name, last name`) of drivers who used the trucks manufactured (`make`) by `DAF` and on the other occasion manufactured by `MAN`.

(4) Find the full names (`first-name, last name`) of drivers who never travelled to `Albany`.

(5) Find the total number of times each truck (`registration`) was used on a trip to `Albany`. There is no need to list the trucks never used on any trip to `Albany`.

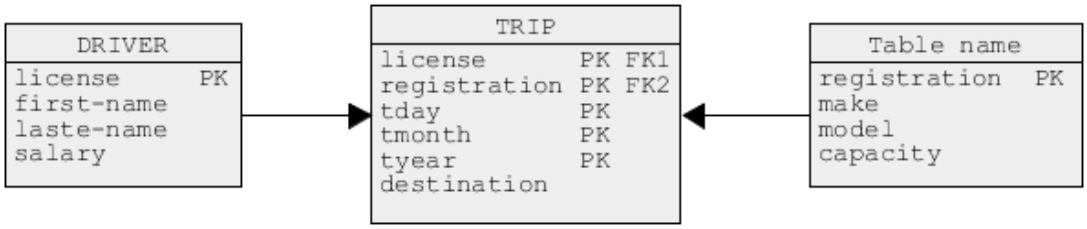
Once completed, copy the entire contents of the `Terminal` window, including data loading outputs, processed queries, ALL messages, and ALL results, to the clipboard. Then, paste these contents into a text file named `solution3.txt`.

Deliverables

A file `solution3.txt` that contains a listing of data loadings and queries performed above, ALL messages and results of operations. A file `solution3.txt` must be created through Copy/Paste of the entire contents of `Terminal` window into a file `solution3.txt`. No screen dumps are allowed and no screen dumps will be evaluated. Solutions that do not include the query processing messages will not receive any marks.

Task 4 (5 marks)
Data processing with Spark

Consider the following logical schema of two-dimensional data cube.



Download a file `task4.zip` published on Moodle together with a specification of Assignment 3 and unzip it. You should obtain a folder `task4` with the following files: `driver.csv`, `truck.csv` and `trip.csv`.

Use a text editor to examine the contents of the files.

Upload the files into HDFS.

Open `Terminal` window and start `pyspark` command line interface to Spark. Use `pyspark` command line interface to implement the following actions. Implementation of each step is worth 1 mark.

(1) Create the schemas for the files `truck.csv`, `driver.csv`, and `trip.csv`.

(2) Create the data frames with the contents of the files `truck.csv`, `driver.csv`, and `trip.csv` using the schemas created in the previous step.

Count the total number of rows in each frame and then list the contents of each frame.

(3) Create an process the following query directly on the trips DataFrame, without creating a temporary view.

Find the total number of times each driver (`license, first name, last name`) travelled to `Albany`. There is no need to list the drivers who never travelled to Albany.

(4) Create a temporary view over a data frame with information about the trips and drivers.

(5) Execute the following query on a temporary view containing information about the trips and drivers.

Find the total number of times each driver (`license, first name, last name`) travelled to `Albany`. There is no need to list the drivers who never travelled to Albany.

When ready, copy into a clipboard the contents of `Terminal` window with the operations processed above and the results listed in the window and paste the results from a clipboard into a text file `solution4.txt`.

Deliverables

A file `solution4.txt` that contains a listing of operations performed above and the results of operations. A file `solution4.txt` must be created through Copy/Paste of the contents of `Terminal` window into a file `solution4.txt`. No screen dumps are allowed and no screen dumps will be evaluated.

Task 5 (5 marks)
Structured stream processing with Spark

Read a description of the following simple data stream processing environment.

A transportation company records all trips completed by its drivers. A trip completion information includes a driver's license number, truck registration number, destination, and total distance travelled. The collected data form a continuous stream of trip records. For example:

```
DRV01,TRK01,Melbourne,1000
DRV02,TRK02,Brisbane,1100
DRV04,TRK04,Adelaide,1200
DRV05,TRK04,Perth,2000
DRV06,TRK03,Canberra,300
… … …
```

An objective of this task is to implement a data stream processing application that displays the total distance travelled by each truck whenever a new trip is completed and a completion records is added to a stream.

For example, after the trips

```
DRV01,TRK01,Melbourne,1000
DRV02,TRK02,Brisbane,1100
DRV04,TRK04,Adelaide,1200
DRV05,TRK04,Perth,2000
DRV06,TRK03,Canberra,300
```

are completed the data stream processing application supposed to return the following values.

```
TRK01,1000
TRK02,1100
TRK03,300
TRK04,3200
```

Use `netcat` utility to simulate a stream of data items given above. It is explained in a specification of laboratory class for week 12 how to use `netcat` utility to simulate a data stream. To do so open `Terminal` window, start `netcat` in Terminal window and enter the quadruples of values like driver's license number, truck registration number, destination, and total distance travelled.

Use a command line interface `pyspark` to implement a data stream processing application.

Testing should be performed in two `Terminal` windows. Open the first Terminal window and start `netcat` utility. Type in a sample data item, for example `DRV01,TRK01,Melbourne,1000`.

Next, open the second `Terminal` window, start `pyspark` and start your data stream processing application. The first results should appear in the second `Terminal` window after a while.

Next, return to the first Terminal window and enter the next data item, for example `DRV02,TRK02,Brisbane,1100`.

Then, the next results should appear in the second `Terminal` window after a while.

Repeat such procedure several times for different trip records.

You can practice a procedure described above in the steps 5, 6, 7, … of a laboratory specification for week 12.

When ready, copy the contents of both `Terminal` windows with a data stream simulated by `netcat` and with a listing of your applications and the results of data stream processing into a file `solution5.txt`.

Deliverables
A file `solution5.txt` that contains a data stream simulated by `netcat` and a listing of your applications and the results of data stream processing. A file `solution5.txt` must be created through Copy/Paste of the contents of `Terminal` windows into a file `solution5.txt`. No screen dumps are allowed and no screen dumps will be evaluated.

Submission of Assignment 3

Note, that you have only one submission. So, make it absolutely sure that you submit the correct files with the correct contents. No other submission is possible !

Submit the files **solution1.txt**, **solution2-1.txt**, **solution2-2.txt**, **solution3.txt**, **solution4.txt** and **solution5.txt** through Moodle in the following way:

(1) Access Moodle at **http://moodle.uowplatform.edu.au/**
(2) To login use a **Login** link located in the right upper corner the Web page or in the middle of the bottom of the Web page
(3) When logged select a site **ISIT912/312 (S225) Big Data Management**
(4) Scroll down to a section **Assessment items (Assignments)**
(5) Click at **In this place you can submit the outcomes of your work on the tasks included in Assignment 3 for ISIT912 students** link.
(6) Click at a button **Add Submission**
(7) Move a file **solution1.txt** into an area **File submissions**. You can also use a link **Add**...
(8) Repeat a step (7) for the files **solution2-1.txt**, **solution2-2.txt**, **solution3.txt**, **solution4.txt** and **solution5.txt**.
(9) Click at a button **Submit assignment.**
(10) Click at the checkbox with a text attached: **By checking this box, I confirm that this submission is my own work, I accept responsibility for any copyright infringement that may occur as a result of this submission, and I acknowledge that this submission may be forwarded to a text-matching service**.
(11) Click at a button **Continue**
(12) Check if **Submission status** is **Submitted for grading**.

End of specification